Université de Bourgogne
L2 Mathématiques Informatique

RAPPORT INFO4B

Base de données Objet

DEHU Gabin
COIPELET Maxence

6 avril 2025

Sommaire

1 Introduction **1**

2 Analyse fonctionnelle **2**
2.1 Objectifs fonctionnels 2
2.2 Scénarios d'utilisation 2
2.3 Découpage en sous-problèmes 3

3 Description des structures de données **4**
3.1 Connexion client/serveur 4
3.2 Gestion de la concurrence et accès aux fichiers JSON 5
3.3 Persistance des objets avec JSON et Jackson 5
3.4 Gestion des comptes utilisateurs 6
3.5 Conversion d'objets avec Jackson 9

4 Spécification des classes principales et leurs méthodes essentielles **9**
4.1 La classe `Serveur` 9
4.2 La classe `GestionnaireClient` 10
4.3 La classe `GestionnaireDonnees` 10
4.4 La classe `GestionnaireCompte` 11
4.5 La classe `ClientLibrairie` 11
4.6 La classe `ClientCMD` 12

5 Architecture logicielle détaillée **13**
5.1 Couche de présentation (Interface utilisateur) 13
5.2 Couche de communication 13
5.3 Couche de données 13
5.4 Couche d'accès aux objets (Persistance) 14

6 Description des algorithmes principaux **15**
6.1 Traitement des commandes client 15
6.2 Conversion en Map<String, Object> avec Jackson 15
6.3 Attribution d'identifiants aux objets 16
6.4 Gestion de la concurrence avec ReadWriteLock 16

7 Jeu de tests **17**
7.1 Test des opérations sur les objets de type `Voiture` en important la librairie dans un code Java 19
7.2 Test des opérations sur les objets de type `Livre` 20
7.3 Validation générale des tests 21

8 Répartition des tâches **21**

9 Conclusion **22**

1 Introduction

Dans le cadre du module "Principes des Systèmes d'Exploitation", nous sommes deux étudiants de deuxième de licence à l'UFR Sciences et Techniques de Dijon et nous avons décidé de réaliser le sujet de projet : "Base de données orientée objet". Ce projet a pour objectif de développer un service complet de gestion de base de données orientée objet en Java. Notre solution permet d'enregistrer, de consulter et de manipuler des collections d'objets enregistrées de manière persistante sur le serveur. Pour améliorer la portabilité des données, nous avons choisi d'utiliser le format JSON – via la bibliothèque Jackson – pour sérialiser les objets, contrairement à une sérialisation binaire en .ser classique.

Le programme se compose de plusieurs modules, chacun distribué sous forme d'un fichier .jar exécutable, ce qui facilite leur déploiement sur différentes machines. Les composants principaux sont :

- **Le serveur** : Il gère l'authentification des utilisateurs, la persistance des objets (les collections d'objets sont stockées sous forme de fichiers JSON) et la communication avec les clients via des sockets TCP. Ces clients sont gérés avec des threads ce qui permet notamment le mode multi-utilisateur sur le serveur. Il intègre des mécanismes de synchronisation (ReadWriteLock) pour garantir la cohérence en cas d'accès concurrent aux données. (`Serveur.java`).
- **Le client en mode CMD** : Il offre une interface en ligne de commande permettant aux utilisateurs de s'inscrire, de se connecter et d'exécuter des commandes CRUD (Create, Read, Update, Delete). Lors de la création d'objets, le client convertit grâce à la librairie client les objets en liste de Map<String, Object> contenant les attributs (String) et leur valeurs (Object) et les envoie au serveur pour enregistrement dans des fichiers JSON nommés d'après le type d'objet (par exemple, `Voiture.json` ou `Livre.json`) (`ClientCMD.java`).
- **La bibliothèque client** : Ce package Java encapsule la logique de communication avec le serveur (connexion, authentification et envoi/réception des commandes et données) et assure la conversion des objets Java en liste de map (et inversement). Cela permet aux clients de réaliser les opérations de manière simple et uniforme (`ClientLibrairie.java`).

Bien que nous ayons développé une interface graphique (ClientGUI), celle-ci n'est présentée dans le corps principal du rapport qu'en bonus dans une section expérimentale, car l'interface en ligne de commande (ClientCMD) constitue l'implémentation principale attendue par le sujet.

Ce rapport, présente tout d'abord une analyse fonctionnelle complète, suivie d'une description détaillée des choix techniques, des structures de données, de l'architecture logicielle et des algorithmes mis en œuvre. Nous y intégrons également des extraits de code et des diagrammes UML pour illustrer notre démarche. Enfin, un jeu de tests démontrant le bon fonctionnement du système ainsi qu'une répartition claire des tâches entre les deux membres du groupe seront proposés à la fin de ce rapport.

2 Analyse fonctionnelle

Cette section détaille les fonctionnalités attendues de l'application, présente les scénarios d'utilisation classiques et décrit le découpage du problème en modules (couches) fonctionnelles. Les choix de conception intègrent désormais la conversion des objets en JSON (à l'aide de Jackson), le packaging en .jar et la mise en place d'un mode de création d'objets via le client CMD.

2.1 Objectifs fonctionnels

Le projet que nous avons développé permet une :

- **Authentification et inscription** :
  - Permettre à des utilisateurs de s'inscrire et de se connecter au serveur.
  - Valider les identifiants via un mécanisme de hachage simple (SHA-256) et stocker les comptes dans un fichier persistant.
  - Limiter le nombre de tentatives d'authentification et gérer la déconnexion après plusieurs échecs.
- **Gestion des objets persistants** :
  - Créer des collections d'objets selon leur type. Lorsqu'un utilisateur crée des objets (via des commandes en mode CMD), ceux-ci sont enregistrés dans des fichiers JSON dédiés (par exemple, `Voiture.json` pour les objets de type Voiture).
  - Permettre la consultation, l'ajout et la suppression d'objets via des commandes CRUD.
- **Communication client-serveur** :
  - Utiliser des sockets TCP pour établir une communication fiable entre clients et serveur.
  - La bibliothèque client gère l'établissement de la connexion, l'authentification et l'échange des données (les commandes et les objets sont transmis sous forme de listes de Map en les convertissant en objets Java et inversement).
  - Le mode principal d'interaction est l'interface en ligne de commande (ClientCMD).
- **Gestion de la concurrence et multi-utilisateurs** :
  - Permettre à plusieurs clients de se connecter simultanément, chaque connexion étant gérée dans un thread dédié.
  - Utiliser des verrous (ReadWriteLock) pour protéger les accès concurrentiels aux fichiers JSON, garantissant ainsi la cohérence des opérations de lecture et d'écriture.

2.2 Scénarios d'utilisation

Pour illustrer le fonctionnement du système, plusieurs scénarios ont été identifiés :

Scénario d'inscription et de connexion

Un utilisateur démarre le client en mode CMD. S'il n'a pas de compte, il choisit l'option d'inscription (l'option numéro 2), saisit un nom d'utilisateur et un mot de passe, qui sont envoyés au serveur pour vérification et enregistrement. Une fois l'inscription validée, l'utilisateur se connecte en envoyant ses identifiants (l'option numéro 1), et le serveur lui répond par un message de bienvenue accompagné d'un autre message indiquant que l'utilisateur s'est connecté avec succès.

Scénario de création et de manipulation d'objets

Après connexion, l'utilisateur peut créer une collection d'objets en spécifiant le type (par exemple, `Voiture`). Les objets sont créés dynamiquement via le client CMD, convertis en listes de Map par la librairie client et envoyés au serveur pour être enregistrés dans un fichier nommé `Voiture.json`. Par la suite, il peut consulter cette collection (commande `LIRE`) ou supprimer des objets spécifiques (commande `SUPPRIMER`).

Scénario multi-utilisateurs

Plusieurs utilisateurs se connectent simultanément au serveur. Chaque connexion est gérée par un thread dédié. Les opérations concurrentes sur les collections d'objets sont synchronisées à l'aide de verrous en écriture et lecture (ReadWriteLock). Par ailleurs, toutes les actions (création, consultation, ajout, suppression) sont loggées avec le nom de l'utilisateur la date dans l'invite de comande du serveur.

2.3 Découpage en sous-problèmes

Pour répondre à l'ensemble des exigences, le projet a été décomposé en plusieurs modules (couches et sous-couches) fonctionnels distincts :

- **Classe d'authentification et de gestion des comptes** : Géré par la classe `GestionnaireCompte`, ce module permet aux utilisateurs de s'inscrire et de se connecter en validant leurs identifiants via un hachage SHA-256. Les comptes sont enregistrés dans un fichier dans comptes/compte.ser.
- **La persistance des données** : Gérée par la classe `GestionnaireDonnees`, ce module est responsable de la sauvegarde et de la récupération des collections d'objets. Contrairement à une sérialisation binaire classique, nous utilisons ici Jackson pour convertir les objets en JSON et les enregistrer dans des fichiers dont le nom correspond au type d'objet. Des verrous (ReadWriteLock) assurent la cohérence des accès concurrents.
- **La communication réseau** : Ce module est basé sur des sockets TCP. La classe `Serveur` écoute en permanence sur un port et délègue la gestion des connexions à la classe `GestionnaireClient`. Du côté client, la bibliothèque `ClientLibrairie` gère l'établissement de la connexion, l'authentification et l'envoi/réception des commandes (les données sont transmises soit sous forme de string, soit en listes de Map).
- **Module de présentation** : L'interface principale est en mode commande (ClientCMD), qui permet aux utilisateurs de saisir des commandes et de visualiser les réponses du serveur.
- **La section gestion de la concurrence** : Afin de permettre l'accès simultané des utilisateurs aux données tout en garantissant leur cohérence, des verrous en lecture/écriture (ReadWriteLock) sont utilisés dans le module de gestion des données sur chaque fichier JSON.

Un diagramme de classes (voir Figure 1) illustre l'interaction entre l'utilisateur et le serveur. On voit alors que la connexion entre le client et le serveur est gérée côté client par la librairie qui permet au client d'ajouter des objets depuis le mode commande ou par du code directement ici par exemple avec testVoiture.

FIGURE 1. Diagramme fonctionnel des cas d'utilisation du système

3 Description des structures de données

Dans notre projet, la persistance et la gestion des objets s'effectuent en enregistrant des fichiers JSON, générés via un objectMapper de la bibliothèque Jackson. Pour assurer la sécurité des accès dans un environnement multi-utilisateur, nous avons également adopté des structures de données thread-safe et des mécanismes de synchronisation.

3.1 Connexion client/serveur

La communication entre le client et le serveur est réalisée via le protocole TCP. Ce choix a été fait pour plusieurs raisons :

— **Fiabilité** : TCP est un protocole orienté connexion qui garantit l'arrivée des données dans le bon ordre et permet la correction d'éventuelles erreurs de transmission. Cela est

important (si ce n'est obligatoire) pour notre application, où la cohérence des commandes (comme la création, la lecture et la suppression d'objets) doit être assurée.
— **Contrôle de flux** : TCP inclut des mécanismes de contrôle de flux, permettant ainsi d'éviter la surcharge du serveur et d'assurer une communication stable même en cas de trafic important.
— **Simplicité d'implémentation** : Étant donné qu'on a besoin pour notre application de la transmission fiable de commandes et de données, TCP s'avère être le choix le plus approprié par rapport à UDP, qui ne garantit pas la fiabilité ni l'ordre des paquets.

En résumé, nous avons opté pour TCP plutôt que UDP, car la fiabilité et l'intégrité des données sont importantes pour garantir le bon fonctionnement de notre système de gestion de données, même si UDP aurait pu être envisagé dans des contextes où la rapidité est préférée à la fiabilité.

3.2 Gestion de la concurrence et accès aux fichiers JSON

Pour permettre à plusieurs clients d'accéder simultanément aux données tout en évitant les conflits lors des opérations de lecture et d'écriture, nous utilisons une **ConcurrentHashMap** pour associer à chaque type d'objet un verrou de type `ReadWriteLock`. Ce verrou permet :

— D'autoriser plusieurs lectures en même temps (grâce au `readLock`).
— D'assurer qu'une opération d'écriture se fasse de manière exclusive (une seule à la fois) (grâce au `writeLock`).

Dans la classe `GestionnaireDonnees.java`, la map des verrous est un attribut de la classe et est déclarée comme suit :

Listing 1. Déclaration des verrous dans GestionnaireDonnees.java

```
private final ConcurrentHashMap<String,
ReadWriteLock> lockMap = new ConcurrentHashMap<>();
```

La méthode `getLock(String type)` vérifie si un verrou existe déjà pour le type donné et le prend et, sinon, en crée un nouveau et il le prend également :

Listing 2. Méthode getLock dans GestionnaireDonnees.java

```
private synchronized ReadWriteLock getLock(String type) {
    if (!lockMap.containsKey(type)) {
        lockMap.put(type, new ReentrantReadWriteLock());
    }
    return lockMap.get(type);
}
```

Ce mécanisme garantit la cohérence lors d'opérations concurrentes sur les fichiers JSON.

3.3 Persistance des objets avec JSON et Jackson

Plutôt que d'utiliser la sérialisation binaire classique, nous convertissons les objets en listes de Map avant de les envoyer au serveur ce qui est nécéssaire car la sérialisation classique s'effectue sur des objets Java que le serveur connaît, c'est à dire pour lesquelles la classe est chargée dans le CLASSPATH, cependant cela impliquerait qu'il faudrait uniquement envoyer des objets de classes fixées au préalable. Avec Jackson, on peut utiliser des objets Java sous forme de listes de Map contenant les attributs de l'objet et leur valeur, ce qui permet de pouvoir les envoyer

au serveur sans qu'il ne connaisse la classe de l'objet pour ensuite le sérialiser dans un fichier JSON. Pour ce faire, nous utilisons l'`ObjectMapper` de la bibliothèque Jackson, configuré pour produire un JSON formaté (indenté) afin de faciliter la lecture.

Dans `GestionnaireDonnees.java`, l'`ObjectMapper` est initialisé dans le constructeur qui liste également tous les types d'objets, donc les fichiers .json présents sur le serveur pour les inclure à la map des verrous car une instance de `GestionnaireDonnees.java` est créée à chaque lancement du serveur.

Listing 3. Initialisation de l'ObjectMapper dans GestionnaireDonnees.java

```
public GestionnaireDonnees(String directory) {
    this.directory = directory;
    File dir = new File(directory);
    if (!dir.exists()) {
        dir.mkdirs();
    }
    this.objectMapper = new ObjectMapper();
    objectMapper.enable(SerializationFeature.INDENT_OUTPUT);
    // Initialisation des verrous pour les fichiers JSON existants
    File[] fichiers = dir.listFiles((d, name)->name.endsWith(".json"));
    if (fichiers != null) {
        for (File fichier : fichiers) {
            String type = fichier.getName().replace(".json", "");
            lockMap.put(type, new ReentrantReadWriteLock());
        }
    }
}
```

La méthode `serialiser` convertit une liste d'objets (représentés sous forme de Map) en JSON et l'écrit dans un fichier :

Listing 4. Sérialisation en JSON dans GestionnaireDonnees.java

```
private void serialiser(File fichier,List<Map<String,Object>>objects)
                        throws IOException{
    objectMapper.writeValue(fichier, objects);
}
```

Inversement, la méthode `deserialiser` lit un fichier JSON et retourne une liste de Map :

Listing 5. Désérialisation depuis un fichier JSON dans GestionnaireDonnees.java

```
private List<Map<String,Object>>deserialiser(File fichier)
    throws IOException{
        return objectMapper.readValue(
            fichier,
            new TypeReference<List<Map<String, Object>>>() {}
        );
}
```

Ces méthodes permettent de stocker les collections d'objets dans des fichiers tels que `Voiture.json` ou `Livre.json`, assurant ainsi la persistance de nos données dans un format lisible et facile à manipuler.

3.4 Gestion des comptes utilisateurs

La gestion des comptes utilisateurs est assurée par la classe `GestionnaireCompte`. Cette classe utilise une structure de données statique pour garantir un stockage unique des comptes, partagé

par toutes les connexions, ce qui permet d'éviter la duplication des données. Les comptes sont stockés dans une `ConcurrentHashMap` pour permettre la gestion de la concurrence et sont sauvegardés dans un fichier de type sérialisé `compte.ser`.

Déclaration et chargement des comptes Le stockage des comptes est réalisé à l'aide d'une variable statique, ce qui permet à tous les threads d'accéder à la même instance. La déclaration est la suivante :

Listing 6. Déclaration de la map des comptes dans GestionnaireCompte.java

```
private static ConcurrentHashMap<String, String> comptes;
```

Le chargement initial des comptes se fait dans le bloc statique, qui vérifie d'abord l'existence du répertoire destiné aux données, puis appelle la méthode `chargerComptes()` :

Listing 7. Bloc statique dans GestionnaireCompte.java

```
static {
    File repertoire = new File(REPERTOIRE_DONNEES);
    if (!repertoire.exists()) {
        repertoire.mkdirs();
    }
    comptes = chargerComptes();
}
```

La méthode `chargerComptes()` vérifie l'existence du fichier de comptes et le charge s'il existe, sinon elle retourne une nouvelle `ConcurrentHashMap` :

Listing 8. Méthode chargerComptes() dans GestionnaireCompte.java

```
private static ConcurrentHashMap<String, String> chargerComptes() {
    if(!new File(REPERTOIRE_DONNEES + "/" + FICHIER_COMPTE).exists()) {
        return new ConcurrentHashMap<>();
    }
    try (ObjectInputStream ois = new ObjectInputStream(
    new FileInputStream(REPERTOIRE_DONNEES + "/" + FICHIER_COMPTE))) {
        return (ConcurrentHashMap<String, String>) ois.readObject();
    } catch (IOException | ClassNotFoundException e) {
        System.out.println("Aucun␣utilisateur␣charg ␣ou␣erreur␣:␣"
        + e.getMessage());
        return null;
    }
}
```

Hachage des mots de passe Pour sécuriser les mots de passe, la méthode `hacherMotDePasse` utilise l'algorithme SHA-256 :

Listing 9. Méthode hacherMotDePasse() dans GestionnaireCompte.java

```
private static String hacherMotDePasse(String motDePasse) {
    try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(motDePasse.getBytes());
        StringBuilder hexString = new StringBuilder();
        for (byte b : hash) {
            hexString.append(String.format("%02x", b));
        }
        return hexString.toString();
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException("Erreur␣de␣hachage", e);
```

```
    }
}
```

Inscription et vérification des comptes L'inscription d'un nouvel utilisateur se fait via la méthode `inscrireUtilisateur`. Cette méthode vérifie d'abord si l'utilisateur existe déjà grâce à `utilisateurExiste()`, puis ajoute le compte (avec le mot de passe haché) et sauvegarde les comptes :

Listing 10. Méthode inscrireUtilisateur() dans GestionnaireCompte.java

```
public static boolean inscrireUtilisateur(String nomUtilisateur,
    String motDePasse) {
    if (utilisateurExiste(nomUtilisateur)) {
        return false; // L'utilisateur existe deja
    }
    comptes.put(nomUtilisateur, hacherMotDePasse(motDePasse));
    return sauvegarderComptes();
}
```

La méthode `utilisateurExiste()` permet de vérifier l'existence d'un compte :

Listing 11. Méthode utilisateurExiste() dans GestionnaireCompte.java

```
public static boolean utilisateurExiste(String nomUtilisateur) {
    return comptes.containsKey(nomUtilisateur);
}
```

La vérification des identifiants lors de la connexion est effectuée par la méthode `verifierIdentifiants()` :

Listing 12. Méthode verifierIdentifiants() dans GestionnaireCompte.java

```
public static boolean verifierIdentifiants(String nomUtilisateur,
    String motDePasse) {
    return comptes.containsKey(nomUtilisateur) &&
           comptes.get(nomUtilisateur).equals(hacherMotDePasse(motDePasse));
}
```

Sauvegarde des comptes La méthode `sauvegarderComptes()` écrit la map des comptes dans un fichier, assurant ainsi la persistance des informations utilisateurs :

Listing 13. Méthode sauvegarderComptes() dans GestionnaireCompte.java

```
private static boolean sauvegarderComptes() {
    try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(
    REPERTOIRE_DONNEES + "/" + FICHIER_COMPTE))) {
        oos.writeObject(comptes);
        return true;
    } catch (IOException e) {
        System.err.println("Erreur␣de␣sauvegarde␣des␣comptes␣:␣"
        + e.getMessage());
        e.printStackTrace();
        return false;
    }
}
```

Résumé Ces extraits montrent comment `GestionnaireCompte` utilise :

— Une variable statique pour garantir un stockage unique des comptes,

— Des mécanismes de lecture/écriture via des flux et des blocs `try-with-resources` pour assurer la bonne gestion des ressources,
— Le hachage SHA-256 pour sécuriser les mots de passe,
— Une méthode de sauvegarde qui persiste les comptes dans un fichier, afin que les informations soient disponibles entre les exécutions du serveur.

L'utilisation de membres statiques dans cette classe est justifiée pour simplifier la gestion des comptes car c'est une fonctionnalité non demandée dans le sujet. En effet, une implémentation avec instanciation d'une classe GestionnaireCompte à chaque lancement du serveur serait une meilleure solution.

3.5 Conversion d'objets avec Jackson

Dans la classe `ClientLibrairie.java`, nous utilisons Jackson pour convertir dynamiquement les objets Java en une représentation sous forme de `Map<String, Object>` avant de les envoyer au serveur. La méthode `convertToMap` effectue cette conversion.

Inversement, la méthode `convertToObject` permet de convertir une Map en un objet Java du type souhaité en s'appuyant sur la représentation en mémoire de la classe compilée, par exemple pour la classe `Voiture` via `Voiture.class`. Cependant les attributs et leurs types de la Map doivent impérativement correspondre aux attributs de la classe dont on donne le `.class` sinon la conversion renverra une erreur.

Ces méthodes facilitent l'échange de données entre le client et le serveur en utilisant un format standardisé (JSON), garantissant ainsi une communication efficace (voir partie 6 pour ces méthodes).

4 Spécification des classes principales et leurs méthodes essentielles

Cette section présente les principales classes du système ainsi que leurs méthodes essentielles. Les extraits de code présentés ici montrent uniquement les parties les plus significatives. Pour le reste, une description détaillée explique le rôle de chaque méthode et la manière dont elles interagissent pour assurer le fonctionnement du système.

4.1 La classe `Serveur`

La classe `Serveur` constitue le point d'entrée du côté serveur. Elle initialise le socket d'écoute serveur, obtient les informations d'hôte, les affiche et attend les connexions entrantes. Pour chaque connexion, elle lance un nouveau thread en instanciant la classe `GestionnaireClient`.

Listing 14. Serveur.java – Méthode main()

```
public static void main(String[] args) {
    System.out.println(getTime() + "Demarrage␣du␣serveur");
    try {
            InetAddress adresseServeur = InetAddress.getLocalHost();
            System.out.println("Nom␣de␣l'hote␣:␣"
            + adresseServeur.getHostName());
            System.out.println("Adresse␣IP␣:␣"
            + adresseServeur.getHostAddress());
```

```
            }catch (UnknownHostException e) {
                System.out.println("Impossible␣d'obtenir␣l'adresse␣IP␣du␣serveur");
                e.printStackTrace();
            }
        try (ServerSocket socketServeur = new ServerSocket(PORT)) {
            System.out.println("Le␣serveur␣ecoute␣sur␣le␣port␣" + PORT);
            GestionnaireDonnees gestionnaireDonnees = new GestionnaireDonnees(
            "data");
            while (true) {
                Socket socket = socketServeur.accept();
                System.out.println(getTime() +
                "␣Nouveau␣client␣en␣cours␣de␣connexion␣depuis␣:␣"
                    + socket.getInetAddress().getHostName());
                new Thread(new GestionnaireClient(socket,
                gestionnaireDonnees)).start();
            }
        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
```

La méthode `getTime()` est une méthode utilitaire statique qui formate la date et l'heure pour les logs serveur.

4.2 La classe `GestionnaireClient`

Cette classe gère la session d'un client connecté au serveur. Elle crée des flux d'entrée/sortie pour communiquer avec la librairie client `ClientLibrairie` et interprète les commandes reçues (comme `CREER`, `LIRE`, `SUPPRIMER`, `LISTER`, etc.).

Description générale :

- **Authentification et inscription** : La méthode `gestionInscription()` gère la procédure d'inscription, tandis que `gestionAuthentification(int nbTentativeAuth)` valide les identifiants en appelant les méthodes statiques de `GestionnaireCompte`.
- **Traitement des commandes** : La méthode `gestionCommande(String commande)` utilise un `switch` sur la commande reçue et appelle les méthodes du module (`GestionnaireDonnees`) pour réaliser l'opération demandée.
- **Fermeture de la connexion** : La méthode `fermer()` assure la libération des ressources en fermant les flux et la socket.

4.3 La classe `GestionnaireDonnees`

La classe `GestionnaireDonnees` gère la persistance des objets. Elle convertit les objets en JSON (via Jackson) et les stocke dans des fichiers dont le nom correspond au type de l'objet (ex. `Voiture.json`). Elle fournit des opérations pour créer, lire, ajouter et supprimer des objets, en utilisant des verrous (ReadWriteLock) pour protéger les accès concurrents.

Comment nous avions dit plus haut dans la troisième section de ce rapport, la méthode `serialiser` convertit une liste d'objets (ici représentés sous forme de Map) en JSON et l'écrit dans un fichier. Cela permet de sauvegarder la collection d'objets de manière persistante.

Les autres méthodes de cette classe (telles que `deserialiser(File fichier)`, `creer(String type, List<Map<String, Object» objects)`, `lire(String type)`, `ajouter(String type, List<Map<String, Object» nouveauxObjets)` et les méthodes de suppression) fonctionnent de manière similaire :

— Elles récupèrent ou créent un verrou associé au type d'objet.
— Effectuent la lecture ou l'écriture du fichier JSON correspondant en utilisant Jackson.
— Assurent une synchronisation adéquate pour éviter les conflits en cas d'accès concurrent.

4.4 La classe `GestionnaireCompte`

Cette classe gère les comptes utilisateurs. Elle stocke les comptes dans une `ConcurrentHashMap` statique qui permet de charger tous les comptes en mémoire. Nous avons décider de procéder comme cela dans le cas ou il n'y aurait pas beaucoup de comptes inscrits pour simplifier la gestion des comptes mais il serait plus pertinent de ne pas charger tous les comptes en mémoire pour garder des ressources du serveur. Les mots de passe sont hachés avec SHA-256 pour plus de sécurité.

Les méthodes `hacherMotDePasse()`, `inscrireUtilisateur(String nomUtilisateur, String motDePasse)`, `utilisateurExiste(String nomUtilisateur)` et `verifierIdentifiants(String nomUtilisateur, String motDePasse)` utilisent cette méthode pour garantir la sécurité des mots de passe lors de l'inscription et de la connexion.

4.5 La classe `ClientLibrairie`

La classe `ClientLibrairie` est un composant central du client dans cette application client-serveur. Elle gère la communication réseau avec le serveur et implémente l'interface `AutoCloseable` pour garantir une fermeture propre des ressources réseau (sockets, flux de données). Elle établit une connexion via des sockets TCP, crée des flux d'objets, et effectue les échanges avec le serveur en convertissant les objets en en Map grâce à Jackson.

Grâce à `AutoCloseable`, la classe peut être utilisée dans un bloc `try-with-resources`, assurant une fermeture automatique des ressources chez la machine de l'utilisateur, par exemple on peut se connecter au serveur dans un code java ayant importé la librairie avec la méthode `getConnection` :

Listing 15. Méthode `getConnection` :

```
public static ClientLibrairie getConnection(String hote, String nomUtilisateur,
String motDePasse) throws IOException, ClassNotFoundException {
    ClientLibrairie client = new ClientLibrairie();
    client.connecter(hote);

    if (!client.authentification(nomUtilisateur, motDePasse)) {
        client.close();
        throw new IOException("Erreur␣:␣Echec␣de␣l'authentification␣pour"+
        + "l'utilisateur␣" + nomUtilisateur);
    }
    return client;
}
```

Listing 16. Exemple d'utilisation de `getConnection` avec `ClientLibrairie` :

```
public static void main(String[] args) {
    try (ClientLibrairie client = ClientLibrairie.getConnection("localhost",
    "nomUtilisateur", "motdepasse")) {
        // Exemple : Lecture des objets de type "Livre" depuis le serveur
        ArrayList<Livre> listeLivres = client.lire("Livre",Livre.class);
        System.out.println("Liste␣des␣livres␣:␣\n" + listeLivres);

    } catch (IOException | ClassNotFoundException e) {
        System.err.println("Erreur␣:␣" + e.getMessage());
    } // Le client est automatiquement ferme ici grace a AutoCloseable
}
```

Cependant on peut également se connecter au serveur avec la méthode `connecter()` ce qui est utilisé dans le client en mode terminal `ClientCMD` mais il ne faut pas oublier de fermer la connection et libérer les ressources avec `close()` dans ce cas. Il est donc préférable de toujours utiliser un bloc `try-with-resources` dans le contexte d'une connexion utilisateur depuis un code Java.

Listing 17. Méthode `connecter()` et `authentification()` dans ClientLibrairie.java

```
public void connecter(String hote) throws IOException {
    InetAddress.getByName(hote);
    socket = new Socket(hote, 8080);
    sortie = new ObjectOutputStream(socket.getOutputStream());
    entree = new ObjectInputStream(socket.getInputStream());
}

public boolean authentification(String nomUtilisateur,
    String motDePasse) throws IOException, ClassNotFoundException {
    sortie.writeObject("AUTHENTIFICATION");
    sortie.writeObject(nomUtilisateur);
    sortie.writeObject(motDePasse);
    sortie.flush();
    String reponseAuth = (String) entree.readObject();
    return "AUTHENTIFICATION␣OK".equals(reponseAuth);
}
```

Les méthodes CRUD de cette classe (comme `creer(ArrayList<Object> listeObjets, String type)`, `lire(String type, Class<T> clazz)`, `ajouter` et `supprimer`) suivent une logique similaire : elles envoient une commande au serveur, transmettent les données converties en Map grâce à Jackson et attendent une réponse indiquant le succès ou l'échec de l'opération.

4.6 La classe `ClientCMD`

Cette classe offre une interface en ligne de commande permettant à l'utilisateur de se connecter, de s'inscrire et d'interagir avec le serveur en envoyant des commandes telles que `LIRE`, `CREER`, `SUPPRIMER`, `LISTER`, etc. Le rôle principal de cette classe est de récupérer l'entrée utilisateur, de formater les commandes et d'afficher les résultats reçus du serveur.

Description :

- La méthode `main()` initialise une instance de `ClientLibrairie` appellée le client qui gère la connection et l'authentification.
- Un menu interactif est présenté à l'utilisateur, qui lui permet de saisir des commandes.
- Les commandes sont découpées en parties pour détecter le type de commande puis elles sont envoyées au serveur par la l'instance de la librairie client `ClientLibrairie`.

5 Architecture logicielle détaillée

Dans le cadre de ce projet, nous avons adopté une conception en couches fonctionnelles, inspirée de l'architecture des systèmes d'exploitation, comme vu lors de ce module "Info4B". Cette approche permet de séparer clairement les responsabilités, facilitant ainsi la maintenance, l'évolution et la réutilisation du code. Les principales couches de notre application sont les suivantes :

5.1 Couche de présentation (Interface utilisateur)

Cette couche est responsable de l'interaction avec l'utilisateur. Elle se charge de recueillir les commandes et d'afficher les résultats. Dans notre projet, nous avons principalement implémenté une interface en ligne de commande, complétée par une interface graphique expérimentale présentée en bonus.

- **ClientCMD.java** : Fournit une interface en ligne de commande permettant aux utilisateurs de saisir des commandes (inscription, connexion, opérations CRUD, etc.) et d'afficher les réponses du serveur.
- **(Bonus) ClientGUI.java** : Une interface graphique développée à titre expérimental pour offrir une alternative à l'interface en mode CMD.

5.2 Couche de communication

Cette couche assure la conversion des objets en Map et principalement le transport des données entre le client et le serveur. Elle masque les détails du protocole réseau, en utilisant des sockets TCP pour garantir une communication fiable.

- **ClientLibrairie.java** (côté Client)
- **GestionnaireClient.java** (côté Serveur)
- **Serveur.java** (pour la connexion initiale)

5.3 Couche de données

Cette couche contient la logique de sérialisation du système. Elle assure l'accès conccurentiel aux données et aux comptes et permet leur sérialisation.

- **GestionnaireCompte.java**
- **GestionnaireDonnees.java**

5.4 Couche d'accès aux objets (Persistance)

Cette couche se charge de l'enregistrement et de la récupération des objets de façon persistante (au format JSON). Elle interagit directement avec le système de fichiers pour lire et écrire les données.

— **GestionnaireDonnees.java** : Outre la logique applicative, cette classe offre un ensemble d'opérations CRUD qui communiquent avec le stockage persistant. Chaque type d'objet est enregistré dans un fichier dédié et des mécanismes de synchronisation garantissent la sécurité des opérations.

Schéma global de l'architecture :
Le schéma ci-dessous illustre l'interaction entre les différentes couches :

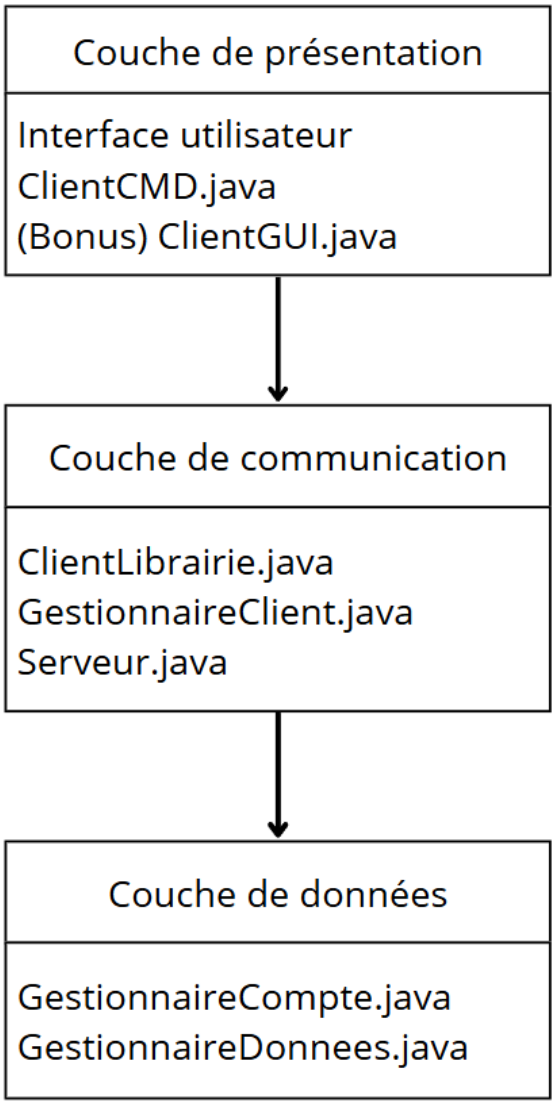

FIGURE 2. Schéma global de l'architecture en couches

Dans ce schéma :

— La **couche de présentation** (ClientCMD et éventuellement ClientGUI) interagit avec l'utilisateur.

— La **couche de communication** (ClientLibrairie et Serveur) assure le transfert des données via TCP.
— La **couche logique métier** (GestionnaireClient, GestionnaireCompte et GestionnaireDonnees) orchestre les opérations d'authentification, de gestion des comptes et des collections d'objets.
— La **couche d'accès aux données** (encore GestionnaireDonnees) s'occupe de la persistance en enregistrant et en récupérant les fichiers JSON.

Cette architecture en couches, inspirée des systèmes d'exploitation, permet de maintenir une séparation claire des responsabilités. Chaque couche offre des services clairement définis aux couches supérieures, facilitant ainsi la maintenance et l'évolution du système.

6 Description des algorithmes principaux

Dans cette section, nous présentons quatre algorithmes clés qui assurent le fonctionnement de notre application. Nous écrivons ici les points essentiels, en ce basant sur les classes et méthodes déjà présentées pour éviter la redondance.

6.1 Traitement des commandes client

L'algorithme de traitement des commandes est central dans la classe `GestionnaireClient`. Une fois qu'un client est authentifié, la méthode `gestionCommande(String commande)` lit la commande envoyée par le client et l'envoie vers l'opération correspondante (par exemple, `CREER`, `LIRE`, `AJOUTER`, `SUPPRIMER`, ou `LISTER`).

— **Fonctionnement :** - Le serveur reçoit une commande sous forme de chaîne via le flux objet. - Un `switch` sur la commande détermine l'opération à réaliser. - Chaque cas du `switch` appelle une méthode de la couche persistance (`GestionnaireDonnees`) pour exécuter l'opération.
— **Référence :** Voir la méthode `gestionCommande(String commande)` dans `GestionnaireClient.java`.

6.2 Conversion en Map<String, Object> avec Jackson

La conversion des objets Java en Map (et inversement) est gérée par la classe `ClientLibrairie` côté client et la classe `GestionnaireDonnees` côté serveur.

— **Fonctionnement :** Pour la communication client-serveur, les objets sont convertis en `Map<String, Object>` à l'aide de la méthode `convertToMap(Object obj)` (implémentée dans `ClientLibrairie`) avant d'être sérialisés en JSON sur le serveur. L'ObjectMapper inspecte la structure de l'objet passé en paramètre (exemple : un objet Livre avec des attributs titre, auteur, pages). Pour ce faire, il utilise la réflexion (reflection), un mécanisme Java permettant d'accéder dynamiquement aux champs (variables d'instance) et aux méthodes getters/setters de l'objet, même s'ils sont privés. Cela permet une analyse générique, sans dépendre d'une structure de classe prédéfinie.

Listing 18. Méthode convertToMap dans ClientLibrairie.java

```
private Map<String, Object> convertToMap(Object obj) {
    return objectMapper.convertValue(obj, Map.class);
}
```

Il y a aussi la méthode `convertToObject` qui convertit des maps en objets en utilisant la classe de l'objet. L'algorithme de ObjectMapper repose sur la réflexion (reflection) et l'analyse du `.class` de la classe cible pour convertir dynamiquement les données d'une Map<String, Object> en une instance d'un objet Java. Lorsqu'on utilise la méthode convertValue(map, clazz), ObjectMapper parcourt les paires clé/valeur de la Map(donc les attributs), puis tente de faire correspondre chaque clé à un champ (attribut) de la classe Java spécifiée par clazz. Il utilise ensuite les types attendus pour ces champs afin de caster ou de convertir les valeurs de la Map vers le type correct, y compris les types imbriqués ou personnalisés.

Listing 19. Méthode convertToObject dans ClientLibrairie.java

```
private <T> T convertToObject(Map<String, Object> map, Class<T> clazz){
    map.remove("type");
    return objectMapper.convertValue(map, clazz);
}
```

- La méthode `serialiser(File fichier, List<Map<String, Object>> objects)` dans `GestionnaireDonnees` utilise Jackson pour écrire ces données dans un fichier JSON du nom du type d'objet à enregistrer. - De même, la désérialisation se fait via la méthode `deserialiser(File fichier)` qui reconstruit la liste de `Map` à partir du fichier JSON.

— **Référence :** Voir les méthodes `serialiser(...)` et `deserialiser(...)` dans `GestionnaireDonnees.java`.

6.3 Attribution d'identifiants aux objets

Dans notre implémentation actuelle, nous n'avons pas de mécanisme pour attribuer un identifiant unique à chaque objet. Pour les opérations de suppression, l'identification des objets se fait en se basant sur leur position dans la liste d'objet enregistrés, comme illustré dans la méthode `supprimer(String type, List<Integer> ids)` de la classe `GestionnaireDonnees` où l'indice `id - 1` est utilisé pour accéder à l'objet à supprimer. C'est un inconvénient principal dans le cadre d'accès concurrentiel car un id peut se retrouver décalé si un autre utilisateur l'a supprimé avant. Nous avons tout de même implémenté le moyen de supprimer des listes d'objets en comparant les attributs donc équivalent à un `.equals()` d'objets Java.

Cette approche se fait sur l'ordre d'apparition des objets dans le fichier JSON. Elle est fonctionnelle pour le moment, mais représente un point d'amélioration pour de futures versions, où l'on pourrait attribuer explicitement un identifiant unique à chaque objet avec des UUID et une commande supprimer qui lirait les données et prendrais le lock jusqu'à la la réponse donc l'envoi de la liste d'ID par l'utilisateur pour une gestion plus robuste des opérations CRUD.

6.4 Gestion de la concurrence avec ReadWriteLock

Pour qu'il y ai une bonne intégrité des données dans un environnement multi-utilisateurs, notre solution se base sur l'utilisation de verrous en lecture/écriture dans la classe `GestionnaireDonnees`.

— **Fonctionnement :** Avant toute opération de lecture, la méthode `lire(String type)` acquiert le `readLock` associé au type d'objet. Cela permet à plusieurs clients de lire simultanément sans interférer. - Lors d'une opération d'écriture (création, ajout, suppression), la méthode correspondante acquiert le `writeLock`, assurant ainsi un accès exclusif pendant la modification des fichiers JSON. - Une fois l'opération terminée, le verrou est immédiatement libéré.

— **Références :** Les méthodes `lire()`, `ajouter()` et `supprimer()` dans `GestionnaireDonnees.java` illustrent ce mécanisme, notamment dans les blocs suivants :

Listing 20. Extrait illustrant l'utilisation du ReadWriteLock dans `lire()`

```
ReadWriteLock lock = getLock(type);
lock.readLock().lock();
try {

    return deserialiser(fichier);
} finally {
    lock.readLock().unlock();
}
```

Ces quatre algorithmes – le traitement des commandes, la conversion en JSON, l'attribution d'identifiants uniques et la gestion de la concurrence – constituent le cœur de notre application. Ils permettent non seulement de réaliser les opérations CRUD de manière fiable, mais également d'assurer la sécurité et la cohérence des données dans un environnement multi-utilisateur (hors suppression par ID).

7 Jeu de tests

On va imaginer que vous n'avez aucune connaissance du sujet et que vous décidez de lancer notre application. On admet pour cela que vous lancez l'application en mode CMD (ClientCMD.jar). Avant cela, il faut d'abord lancer le serveur (écrire `"java -jar nomFichier.jar` dans un terminal ouvert depuis le dossier du fichier que l'on veut ouvrir, ici `Serveur.jar`).

```
[2025-03-20 18:46:43] Démarrage du serveur
Nom de l'hôte : <nom de l'appareil utilisé>
Adresse IP : <adresse IP de l'appareil>
Le serveur écoute sur le port 8080
```

Maintenant on peut désormais lancer le fichier `ClientCMD.jar` pour se conncecter au serveur. Plusieurs requêtes à la suite vont être posées au client : d'abord d'entrer son adresse, puis de choisir entre 1 et 2 en fonction de s'il veut s'inscrire ou se connecter. Si l'utilisateur choisit "1", son nom d'utilisateur et son mot de passe vont lui être demandé et un message apparaitra pour lui signaler que l'inscription a réussie, ou un message d'erreur si l'utilisateur s'est mal inscrit ou s'il est déjà inscrit.

```
Entrez l'adresse de l'hôte (ex: localhost) : localhost
Tapez INSCRIPTION (1) ou LOGIN (2) : 1
Entrez le nom d'utilisateur : Jean
Entrez le mot de passe : mdp21
Inscription réussie pour Jean
Tapez INSCRIPTION (1) ou LOGIN (2) :
```

Une fois que l'utilisateur Jean s'est inscrit avec comme mot de passe "mdp21" (mot de passe peu recommandé), l'utilisateur peut se connecter en tapant "2", il va devoir retaper son nom d'utilisateur et son mot de passe pour se connecter.
Une fois connecté, la liste de toutes les commandes possible s'affiche avec leur description. L'utilisateur peut choisir de créer un objet, d'en supprimer un, de lister tous les objets présents sur le serveur, de l'aide si il veut réafficher cette liste de commande ou enfin stopper la session en fermant la connexion serveur :

```
-----------------------------------------------------
Commande : LIRE <nomObjet>
-----------------------------------------------------
Description :
  Permet de récupérer et d'afficher les objets enregistrés correspondant
  au nom fourni.
-----------------------------------------------------
Commande : CREER <nomObjet>
-----------------------------------------------------
Description :
  Crée un ou plusieurs objets du type spécifié avec des attributs
  personnalisables (String, int, double ou boolean).
-----------------------------------------------------
Commande : SUPPRIMER <nomObjet> <ID1> <ID2> ...
-----------------------------------------------------
Description :
  Supprime les objets du type spécifié dont les identifiants (ID)
  correspondent aux valeurs indiquées.
-----------------------------------------------------
Commande : LISTER
-----------------------------------------------------
Description :
  Liste tous les types d'objets présents sur le serveur.
-----------------------------------------------------
Commande : AIDE
-----------------------------------------------------
Description :
  Affiche ce menu.
-----------------------------------------------------
Commande : STOP
-----------------------------------------------------
Description :
  Termine la session en fermant la connexion avec le serveur.
-----------------------------------------------------
```

Pour l'instant, aucun n'objet n'est créee donc LIRE et LISTER ne renvoient rien :

```
LISTER
Aucun objet enregistré sur le serveur.
```

Il faut donc créer un objet. L'utilisateur doit donc saisir la commande "CREER" suivi du nom de l'objet (sensible à la casse sous Linux). Il va donc ensuite devoir créer les attributs de l'objet avec leur nom et type. Une fois les attributs faits, l'utilisateur doit saisir "fin" pour ensuite devoir rentrer les valeurs de chaque attribut.
Un message demandant si l'utilisateur apparaitra pour lui demander si il veut créer d'autres objets de ce type, si il saisit non, il est affiché qu'un objet de ce type a bien été crée.

```
CREER table
Création d'objets de type: table
Nom de l'attribut (ou 'fin' pour terminer la définition): surface
Type de l'attribut (String, int, double, boolean): int
```

```
Nom de l'attribut (ou 'fin' pour terminer la définition): epaisseur
Type de l'attribut (String, int, double, boolean): double
Nom de l'attribut (ou 'fin' pour terminer la définition): fin

--- Saisie des valeurs pour un nouvel objet ---
surface (int): 127
epaisseur (double): 10
Voulez-vous créer un autre objet de ce type? (oui/non): non
Création de 1 table réussie.
```

Maintenant qu'un objet est crée, on peut utiliser la commande `LISTER` et la commande `LIRE`. Voici ce que cela fait :

```
LISTER
Types disponibles : table
LIRE table
Lecture de(s) objet(s) de type table en String.
[ {
  "id" : 1,
  "surface" : 127,
  "epaisseur" : 10.0
} ]
```

On peut désormais également utiliser la commande `SUPPRIMER` pour supprimer la table que l'on vient de créer :

```
SUPPRIMER table 1
Suppression de 1 table.
```

7.1 Test des opérations sur les objets de type `Voiture` en important la librairie dans un code Java

Il y a une deuxième manière de faire des opérations dans le mode CMD, c'est d'utiliser la classe la classe `TestVoiture.java` (`clientTestVoiture.jar`) qui est un exemple de code que pourrait écrire un utilisateur pour se connecter au serveur et enregistrer des objets. Il effectue les tests suivants :

- **Création :** Une liste d'objets `Voiture` est créée et envoyée au serveur via la méthode `creer()` de la bibliothèque client. Le serveur enregistre cette collection dans le fichier `Voiture.json`.
- **Lecture :** La commande `lire("Voiture", Voiture.class)` est utilisée pour récupérer la collection et vérifier que tous les objets ont été correctement enregistrés.
- **Suppression :** En spécifiant une liste d'indices (par exemple, 1 et 3), la commande `supprimer("Voiture", listeId)` supprime les objets correspondants. La lecture suivante confirme la suppression.
- **Ajout :** Un objet supplémentaire est ajouté avec la méthode `ajouter()`, et la lecture de la collection démontre que l'ajout a été effectué.

Ces opérations permettent de vérifier que le serveur gère correctement la persistance des données et que les fichiers JSON sont mis à jour de manière cohérente. Pour lancer cette classe, il faut trois

arguments supplémentaires : l'adresse de l'utilisateur (`"localhost"` ici), le nom d'utilisateur et le mot de passe de l'utilisateur :

```
 java -jar clientTestVoiture.jar localhost Jean mdp21
Authentification réussie pour Jean
Création de 4 Voiture réussie.
Lecture de 4 objet(s) de type Voiture
Résultat création : [Voiture{nom='3008',
marque='Peugeot', nbChevaux=130, nbPlaces=7}, Voiture{nom='Swift',
marque='Suzuki', nbChevaux=90, nbPlaces=4}, Voiture{nom='Yaris',
marque='Toyota', nbChevaux=120, nbPlaces=5}, Voiture{nom='Corolla',
marque='Toyota', nbChevaux=140, nbPlaces=5}]
Suppression de 2 Voiture.
Lecture de 2 objet(s) de type Voiture
Résultat suppression premier et troisième : [Voiture{nom='Swift',
marque='Suzuki', nbChevaux=90, nbPlaces=4}, Voiture{nom='Corolla',
marque='Toyota', nbChevaux=140, nbPlaces=5}]
Suppression de 2 Voiture.
Erreur : Aucun objet de type Voiture n'est enregistré sur le serveur.
Résultat suppression avec liste initiale de voitures : []
Ajout de 1 Voiture réussie.
Lecture de 1 objet(s) de type Voiture
Résultat ajout 1 voiture : [Voiture{nom='Picanto', marque='Kia',
nbChevaux=80, nbPlaces=5}]
Ajout de 4 Voiture réussie.
Lecture de 5 objet(s) de type Voiture
Résultat ajout 4 voiture : [Voiture{nom='Picanto', marque='Kia',
nbChevaux=80, nbPlaces=5}, Voiture{nom='3008', marque='Peugeot',
nbChevaux=130, nbPlaces=7}, Voiture{nom='Swift', marque='Suzuki',
nbChevaux=90, nbPlaces=4}, Voiture{nom='Yaris', marque='Toyota',
nbChevaux=120, nbPlaces=5}, Voiture{nom='Corolla', marque='Toyota',
nbChevaux=140, nbPlaces=5}]
```

On voit donc bien que toutes les opérations se produisent. C'est une autre manière de voir que l'on peut créer et manipuler des objets java en mode CMD.

7.2 Test des opérations sur les objets de type `Livre`

Il est également possible d'effectuer des opérations CRUD en exécutant la classe `clientTestLivre.jar`, qui va lire des objets de type `Livre` qui peuvent être crées au préalable en mode CMD (en faisant des attributs exacts à ceux de la classe donc `String nom / String auteur / double prix`) pour montrer qu'on peut bien obtenir des objets Java à partir d'objets créees en CMD.

La classe `TestLivre.java` se concentre sur la lecture d'objets :

— **Lecture :** Elle utilise la commande `lire("Livre", Livre.class)` pour récupérer et afficher la collection d'objets de type `Livre` qui ont été créés précédemment en mode commande.

Ce test a pour but de valider la capacité du système à restituer correctement des objets d'un autre type.

7.3 Validation générale des tests

L'ensemble des tests a permis de confirmer que :

- Les opérations CRUD (Création, Lecture, Ajout, Suppression) fonctionnent correctement.
- La communication entre le client et le serveur est fiable et respecte le protocole défini.
- La synchronisation via des verrous (ReadWriteLock) dans la classe `GestionnaireDonnees` assure la cohérence des données en situation de concurrence.

Les résultats de ces tests sont affichés sur la console, et des extraits de sortie montrent, par exemple, le nombre d'objets créés, supprimés ou ajoutés. Ces tests constituent une validation complète du fonctionnement de l'application et de son respect des consignes.

8 Répartition des tâches

Dans le cadre de ce projet, les tâches ont été réparties de manière plutôt équitable entre les deux membres du groupe, Maxence et Gabin, malgré la difficulté du projet.

Le travail de Gabin

- **Gestion des données et persistance :** Gabin a principalement travaillé sur la partie données, en développant la classe `GestionnaireDonnees` et `ClientLibrairie`. Il a mis en place la conversion des objets en JSON via Jackson et la persistance de ces données dans des fichiers (par exemple, `Voiture.json` ou `Livre.json`). Il a également assuré la gestion de la concurrence à l'aide des verrous (ReadWriteLock) pour garantir la cohérence des opérations de lecture et d'écriture.
- **Tests et validation :** Il a réalisé les classes de test `TestVoiture.java` et `TestLivre.java` afin de valider les opérations CRUD et la persistance des données.
- **Gestion des comptes utilisateurs :** Gabin a implémenté la classe `GestionnaireCompte`, qui gère l'inscription et la vérification des comptes utilisateurs. Pour cela, il a utilisé une `ConcurrentHashMap` pour stocker les comptes et un mécanisme de hachage SHA-256 pour sécuriser les mots de passe.

Le travail de Maxence

- **Communication et logique serveur/client :** Maxence a développé la partie serveur et client. Il a créé la classe `Serveur` qui initialise le `ServerSocket` et gère les connexions entrantes, ainsi que la classe `GestionnaireClient` qui interprète et traite les commandes envoyées par les clients.
- **Interface en ligne de commande (ClientCMD) :** Il a conçu et implémenté la classe `ClientCMD`, qui fournit une interface textuelle pour la saisie des commandes (telles que `LIRE`, `CREER`, `SUPPRIMER`, etc.) et la communication avec le serveur via la bibliothèque `ClientLibrairie`.

Difficultés rencontrées et solutions apportées

- **Difficultés de Gabin :**
  - *Gestion de la concurrence dans la persistance des données :* La mise en place des verrous avec `ReadWriteLock` pour sécuriser les accès aux fichiers JSON s'est révélée complexe. Gabin a réussi à faire cela en testant chaque opération CRUD et en ajustant l'utilisation des verrous pour garantir l'intégrité des données.

- *Sécurisation et gestion des comptes utilisateurs :* Le développement de la classe `GestionnaireCompte` a nécessité de sécuriser les mots de passe via SHA-256 et d'assurer une persistance fiable des comptes. Gabin a résolu ces problèmes en utilisant des structures de données thread-safe et en adoptant le modèle `try-with-resources` pour gérer correctement les flux.

- **Difficultés de Maxence :**
  - *Mise en place de la communication réseau :* La gestion des connexions via les sockets TCP et la conception du protocole de communication entre le client et le serveur ont représenté de légère difficultés mais rien de très insurmontable, sachant que la plus grande difficulté du projet résidait dans la création et sérialisation des objets Java.
  - *Traitement des commandes en mode CMD :* La conception de l'interface en ligne de commande, ainsi que la séparation et le découpage des commandes en segments pour une interprétation correcte par le serveur, ont demandé plusieurs itérations. Maxence a intégré des messages d'erreur clairs et une validation systématique des entrées pour garantir un fonctionnement fluide.

9 Conclusion

En conclusion, la répartition des tâches entre Maxence et Gabin a été équilibrée et complémentaire. Chaque membre a pu se concentrer sur des domaines spécifiques tout en collaborant pour intégrer l'ensemble des modules du système. Cette collaboration a permis de surmonter les difficultés techniques rencontrées et d'assurer la cohérence et la robustesse de l'application.

Ce projet a été compliqué, chronophage et éprouvant mais nous a permis de découvrir un nouveau de type de code à programmer. Nous avons appris comment fonctionne un serveur réseau et les connexions liées. Un grand merci à nos professeurs qui nous ont accompagnés dans ce module et nous ont fait découvrir ce type d'application.

Section Bonus : Interface Graphique Expérimentale

En complément de l'interface en ligne de commande (CMD) qui constitue le mode principal de notre application, nous avons expérimenté une interface graphique (GUI). Cette expérimentation s'inspire du module « Interface visuelles » suivi en Info2B l'année dernière, et avait pour objectif de reformater le mode CMD en une interface plus agréable, pour voir comme cela pourrait rendre.

Pour cette expérimentation, nous avons utilisé la bibliothèque **Java Swing** pour construire l'interface, ainsi que des thèmes modernes tels que **FlatLaf** afin d'obtenir un rendu visuel attrayant et un mode sombre cohérent avec l'identité graphique choisie pour l'application.

Remarque : Cette interface GUI a été développée à titre expérimental et n'est pas obligatoire pour le fonctionnement du système principal. Elle illustre notre volonté d'explorer différentes approches d'interfaces utilisateurs et d'apporter une amélioration esthétique à l'expérience utilisateur, sans compromettre l'efficacité du mode de commande textuel.